SECURITY AUDIT OF

OP_NET Rust Merkle Tree

Private Report

Version: 1.0

February 12, 2026

Verichains Lab

info@verichains.io

https://www.verichains.io

EXECUTIVE SUMMARY

This Security Audit Report was prepared by Verichains Lab on **February 12, 2026**. We would like to thank the OP_NET Team for trusting Verichains Lab in auditing smart contracts. Delivering high-quality audits is always our top priority.

This audit focused on identifying security flaws in code and the design of the OP_NET Rust Merkle Tree. The scope of the audit is limited to the source code files provided to Verichains. Verichains Lab completed the assessment using manual, static, and dynamic analysis techniques.

During the audit and penetration testing process, the audit team had identified some issues in the source code.

TABLE OF CONTENTS

1. MANAGEMENT SUMMARY 4
   1.1. About OP_NET Rust Merkle Tree 4
   1.2. Audit Methodology 4
   1.3. Disclaimer 4
   1.4. Acceptance Minute 5
2. AUDIT OVERVIEW 6
   2.1. Audit Scope 6
   2.2. Overview 6
3. FINDINGS 9
   3.1. Ensuring Proper Usage of `verify()` to Prevent Malicious Proof Exploitation `[CRITICAL]` `[FIXED]` 9
   3.2. Hash Collision Risk When `sort=false` `[CRITICAL]` `[FIXED]` 11
   3.3. Disruption of Merkle Proof Integrity Due to Leaf Sorting `[CRITICAL]` `[FIXED]` 13
   3.4. `get_index_by_hash` is ambiguous for duplicate leaves `[INFO]` 14
4. VERSION HISTORY 15

1. MANAGEMENT SUMMARY

1.1. About OP_NET Rust Merkle Tree

This project aims to develop a high-performance, secure Merkle tree library in Rust, with seamless integration for both Rust and Node.js applications. The library leverages SHA-256 as its cryptographic hash function and is designed to be secure against known vulnerabilities, ensuring robust state proof validations. Optimized for both performance and security, this library is intended for use in applications that require strong data integrity and validation mechanisms.

1.2. Audit Methodology

Our security audit process includes two steps:

- Static Analysis: automated security scanning using Rust security tooling.
- Manual Code Review: the codebase are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities and security considerations that were examined during the audit:

- Memory Safety Issues
- Integer Safety
- Concurrency Issues
- Denial of Service (DoS)
- Cryptographic Vulnerabilities
- Unsafe Code Blocks
- FFI Boundary Issues
- Input Validation
- Logic Flaws
- Access Control

For vulnerabilities, we categorize the findings into categories as listed in table below, depending on their severity level:

Severity Level	Description
CRITICAL	A vulnerability that can disrupt the contract functioning; creates a critical risk to the contract; required to be fixed immediately.
HIGH	A vulnerability that could affect the desired outcome of executing the contract with high impact; needs to be fixed with high priority.
MEDIUM	A vulnerability that could affect the desired outcome of executing the contract with medium impact in a specific scenario; needs to be fixed.
LOW	An issue that does not have a significant impact, can be considered as less important.

1.3. Disclaimer

OP_NET Team acknowledges that the security services provided by Verichains, are conducted to the best of their professional abilities but cannot guarantee 100% coverage of all security vulnerabilities. OP_NET Team understands and accepts that despite rigorous auditing, certain vulnerabilities may remain undetected. Therefore, OP_NET Team agrees that Verichains shall not be held responsible or liable, and shall not be charged for any hacking incidents that occur due to security vulnerabilities not identified during the audit process.

1.4. Acceptance Minute

This final report served by Verichains to the OP_NET Team will be considered an Acceptance Minute. Within **seven** days, if no any further responses or reports is received from the OP_NET Team, the final report will be considered fully accepted by the OP_NET Team without the signature.

2. AUDIT OVERVIEW

2.1. Audit Scope

The audit focus on identifying security flaws in the code of OP_NET Rust Merkle Tree. It was first conducted on commit `5b6435215eb5ab5d1da57905eae1c5fc1ca0ea27` from git repository `https://github.com/btc-vision/rust-merkle-tree`. Latest audited commit is `6b3bcbb69f2c02b319e4d2ce3cc71933455d143d`.

This table summarizes the report revision history, mapping each report version to its corresponding Git commit hash.

Report Version	Git Commit
0.1	`5b6435215eb5ab5d1da57905eae1c5fc1ca0ea27`
0.2	`6b3bcbb69f2c02b319e4d2ce3cc71933455d143d`
1.0	`6b3bcbb69f2c02b319e4d2ce3cc71933455d143d`

2.2. Overview

This project is a high-performance Merkle tree library written in Rust with Node.js bindings via N-API. It uses SHA-256 as its cryptographic hash function and is designed for blockchain state proof validations in the OPNet ecosystem. The library enables the construction of Merkle trees from arbitrary data, the generation of inclusion proofs for individual leaves, and the verification of those proofs against a known root hash.

2.2.1. Hashing Strategy

The library uses SHA-256 exclusively, with two distinct hashing modes:

- **Leaf Hashing**: Each leaf is double-hashed using SHA-256 — `SHA256(SHA256(data))` — consistent with Bitcoin's convention. This approach mitigates length extension attacks and aligns with Bitcoin's transaction ID computation. Both tree variants (`Sha256Normal` and `Sha256Canonical`) share an identical `hash_leaf` implementation.
- **Internal Node Hashing**: Parent nodes are computed by concatenating two child hashes and applying a single SHA-256 pass. The concatenation order differs by variant: `Sha256Normal` preserves the left-then-right tree position, while `Sha256Canonical` always places the lexicographically smaller hash first (`min || max`), ensuring order independence.

The hash logic is abstracted behind a `HashMethod` trait that defines `hash_leaf(data) → Vec<u8>` and `hash_nodes(left, right) → Vec<u8>`. This trait-based design allows the same tree construction and proof logic to be reused across both variants without code duplication.

2.2.2. Tree Variants

The library provides two tree variants, each suited to different use cases:

Normal Tree (`MerkleTree`)

The Normal variant preserves the original insertion order of leaves. Leaves are placed in the flattened array in the exact order they are provided, and internal nodes hash their children in positional order (left child first, then right child). This means that reordering the same set of leaves produces a different root hash.

Each proof step in the Normal variant carries a direction bit (`is_left`) that records whether the sibling sits on the left or right of the current node. During verification, the direction bit determines the concatenation

order: if the sibling `is_left`, it is placed before the current hash; otherwise, it is placed after. This makes proof verification sensitive to the original leaf ordering and protects against sibling-swap attacks where an adversary attempts to flip the hash order to forge a valid proof.

Canonical Tree (`MerkleTreeCanonical`)

The Canonical variant sorts all leaves by raw byte comparison (`Vec<u8>::sort()`) before placing them in the tree. Internal nodes always hash their children in sorted order — the smaller hash is concatenated first — eliminating dependence on insertion order. The same set of leaves always produces the same root, regardless of the order in which they are supplied.

Canonical proofs do not require direction bits. Since `hash_nodes` in `Sha256Canonical` always places the smaller input first, the verifier can reconstruct the correct parent hash from any pair of child hashes without knowing their original positions. This simplifies the proof format: each step is a plain 32-byte sibling hash rather than a `(bool, hash)` pair.

The Canonical variant is particularly useful in scenarios where leaf ordering cannot be guaranteed across different nodes or systems, such as blockchain state proofs where transaction ordering may vary by implementation.

2.2.3. Tree Construction

Tree construction follows these steps in both variants:

1. **Parallel Leaf Hashing**: Raw leaf data is double-hashed in parallel using Rayon's `par_iter()`, producing a vector of 32-byte leaf hashes. For the Canonical variant, these hashes are then sorted.
2. **Array Allocation**: A flattened array of size `2n - 1` is allocated, where `n` is the number of leaves. This binary-heap-style layout stores the root at index 0 and leaves in the bottom half (indices `n-1` through `2n-2`).
3. **Leaf Placement**: The hashed leaves are placed into the bottom half of the array, each protected by a `Mutex` to support concurrent access during the build phase.
4. **Recursive Fork-Join Build**: Internal nodes are built from the root downward. At each node, Rayon's `join()` is used to recursively compute the left and right subtrees in parallel. Once both children are resolved, the parent hash is computed and stored. This approach maximizes CPU utilization by distributing subtree computation across available threads.

The construction rejects empty leaf sets with an explicit error. After building, the `Mutex` wrappers are unwound into a plain `Vec<Vec<u8>>`, and the root hash is cached for fast retrieval.

2.2.4. Proof Generation and Verification

Proof Generation

Starting from a target leaf's index in the flattened array, the algorithm walks upward toward the root. At each level, it computes the sibling index using binary-heap arithmetic (`sibling_index`, `parent_index`) and records the sibling's hash. For the Normal variant, it also records whether the sibling is on the left (`is_left = sibling < node`).

Once the path is fully assembled, the proof is self-verified against the tree's stored root before being returned. If verification fails — indicating a corrupted tree or an index pointing to an internal node rather than a leaf — the function returns an error rather than an invalid proof. This self-verification step prevents the library from ever emitting a proof that would fail external verification.

Proof Verification

Verification takes a proof, a leaf hash, and an expected root. The verifier folds the proof steps from the leaf hash upward, computing each parent hash by combining the current hash with the sibling hash from the proof step. For Normal proofs, the `is_left` flag determines concatenation order. For Canonical proofs, the `hash_nodes` function automatically sorts the two inputs. The final computed hash is compared against the expected root; equality confirms the leaf's inclusion.

2.2.5. Key Features

- **SHA-256 Hashing**: Double SHA-256 for leaves, single SHA-256 for internal nodes. The hash logic is abstracted behind a `HashMethod` trait, with `Sha256Normal` and `Sha256Canonical` as concrete implementations.
- **Two Variants**: Normal (order-preserving) and Canonical (order-independent) trees, each with a matching proof type. The Normal variant stores `(bool, Vec<u8>)` proof steps while the Canonical variant stores only sibling hashes.
- **Cross-Platform**: Written in Rust with N-API bindings, enabling seamless use from Node.js applications.
- **Parallel Construction**: Uses Rayon for data-parallel leaf hashing and concurrent subtree building, scaling efficiently with available CPU cores.
- **Self-Verified Proofs**: Every generated proof is verified against the tree's root before being returned, ensuring the library never emits an invalid proof.
- **Immutable After Construction**: The tree exposes no mutation API once built. There is no method to add, remove, or modify leaves after construction, preventing post-construction tampering.
- **Error Handling**: Operations that can fail — empty leaf sets, out-of-range indices, missing hashes, mutex poisoning — return `Result` types with descriptive error messages rather than panicking.

3. FINDINGS

3.1. Ensuring Proper Usage of `verify()` to Prevent Malicious Proof Exploitation `[CRITICAL]` `FIXED`

SCOPE

- Commit: `5b6435215eb5ab5d1da57905eae1c5fc1ca0ea27` .

DESCRIPTION

The usage of the `verify()` function must be specified as `proof.verify(root, MerkleTree.hash(value))` rather than `proof.verify(root, hash)` with an arbitrary hash. Incorrect usage of the `verify()` function can allow a malicious prover to truncate a valid proof and use it as evidence for a node's hash that does not belong to the leaves of the tree. The following test case demonstrates this vulnerability:

```rust
#[test]
fn test_1() {
    let hashes = [
        "93ca042c86dafda63e1a03b4625f614dc9364231e986d72490a480a4cc591c4a",
        "713af6f789258961f0ed63b4073060bae6ba7e8b92bcb5383ebe18e6e76289d6",
        "5d351e5962324a1b9920278825ca07b94d020b34941d20d5ac0f44dbbf3a5258",
        "493c543220bceffa21283b176955173baa7745d563a7b5e2cae0b4253419a87f",
        "39e429c0920f4089a43dbe24a7dfcfe0552bdaabfcc9356cde88f9ea18972bf4",
        "33b544b8002201957eaa0816c4ee2bc244d4cce765e599df6f25adbb2cdb0c08",
        "2e3580210116e2ab7c7705f8f0b6217ac7a0ac5f31de892f847171c073a7542f",
        "016a2b39a42811f88585a5e07ee3a57283607b47816945d61354577ff8868378",
        "c67892017db365f15687b283fea0741145e1b54a62430fd814e1755c6e25949e",
        "af4f1bfe5c512e9265718f3fa9a028f7e70b29860c705763ecbb541f4a5877ce",
        "a0e25bb110b98aa9c3e2be61e432cb432788d0dbe29991acd8ced7c7d9386aea",
        "96ca5d6526e42a2a9da666e27cd8332c3a6b4cada4561726401f66ba08eaaa42",
    ];
    let root = "1eb2fbe0d23ed86d1ad0da939771e8320da2c7de2c341960fe854a7f1ee317c4";

    let tree = MerkleTreeSha256::from_leaves_hashes(
        hashes.iter().map(|h| hex::decode(h).unwrap()).collect(),
        true
    ).unwrap();
    assert_eq!(tree.get_root().unwrap(), hex::decode(root).unwrap());

    // Create a proof for one of the leaf nodes
    let proof = MerkleProofInner::<MerkleTreeSha256>::new_from_proof(
        [
            "713af6f789258961f0ed63b4073060bae6ba7e8b92bcb5383ebe18e6e76289d6",
            "9f5040eb0ee98927dcf3e7256161cad8a55f1530c6106fdc75c674c98d6cb8fe",
            "4d42ca27311b1512c3d3cd5ac07864264b096981cbc8b19bef642613023ca132",
            "9e54701031c343fbf4d2848a4de7df9252a1bac6b4e4b83e64d14ac44c070e4e",
        ]
            .iter()
            .map(|h| hex::decode(h).unwrap())
            .collect()
    );

    // Verify the proof with the correct leaf hash
    assert!(proof.verify(
        &hex::decode(root).unwrap(),
        &hex::decode("93ca042c86dafda63e1a03b4625f614dc9364231e986d72490a480a4cc591c4a").unwrap()
    ));

    // Create a proof with a truncated set of hashes
    let proof = MerkleProofInner::<MerkleTreeSha256>::new_from_proof(
        [
            "9f5040eb0ee98927dcf3e7256161cad8a55f1530c6106fdc75c674c98d6cb8fe",
            "4d42ca27311b1512c3d3cd5ac07864264b096981cbc8b19bef642613023ca132",
            "9e54701031c343fbf4d2848a4de7df9252a1bac6b4e4b83e64d14ac44c070e4e",
        ]
            .iter()
            .map(|h| hex::decode(h).unwrap())
            .collect()
    );

    // Verify the proof with a fake leaf hash
    assert!(proof.verify(
        &hex::decode(root).unwrap(),
        &hex::decode("aae47106d882563487de43ea5c0ac5ec53a60e2d3cc9a88f93b0d33cf0c78ddc").unwrap()
```

```
    ));
}
```

RECOMMENDATION

Correct the usage of `proof.verify` function.

UPDATE

This issue was fixed in commit `6b3bcbb69f2c02b319e4d2ce3cc71933455d143d` .

3.2. Hash Collision Risk When `sort=false` [CRITICAL] FIXED

SCOPE

- Commit: `5b6435215eb5ab5d1da57905eae1c5fc1ca0ea27`

DESCRIPTION

The `hash_nodes` function combines two input nodes (e.g., A and B) and sorts them before hashing to ensure deterministic results. However, sorting removes positional context, meaning the order of the child nodes (A, B) and (B, A) becomes indistinguishable after hashing. Both configurations produce the same hash, making the tree vulnerable to hash collision attacks.

When the sort parameter is set to false, the tree construction retains the original input order. An attacker can rearrange the nodes in the tree and produce a valid proof, even though the structure has been altered, as the hash computation does not distinguish node positions.

The following test case demonstrates this vulnerability:

```rust
#[test]
fn test_2() {
    // Original set of hashes
    let hashes_1 = [
        "93ca042c86dafda63e1a03b4625f614dc9364231e986d72490a480a4cc591c4a",
        "713af6f789258961f0ed63b4073060bae6ba7e8b92bcb5383ebe18e6e76289d6",
        "5d351e5962324a1b9920278825ca07b94d020b34941d20d5ac0f44dbbf3a5258",
        "493c543220bceffa21283b176955173baa7745d563a7b5e2cae0b4253419a87f",
        "39e429c0920f4089a43dbe24a7dfcfe0552bdaabfcc9356cde88f9ea18972bf4",
        "33b544b8002201957eaa0816c4ee2bc244d4cce765e599df6f25adbb2cdb0c08",
        "2e3580210116e2ab7c7705f8f0b6217ac7a0ac5f31de892f847171c073a7542f",
        "016a2b39a42811f88585a5e07ee3a57283607b47816945d61354577ff8868378",
        "c67892017db365f15687b283fea0741145e1b54a62430fd814e1755c6e25949e",
        "af4f1bfe5c512e9265718f3fa9a028f7e70b29860c705763ecbb541f4a5877ce",
        "a0e25bb110b98aa9c3e2be61e432cb432788d0dbe29991acd8ced7c7d9386aea",
        "96ca5d6526e42a2a9da666e27cd8332c3a6b4cada4561726401f66ba08eaaa42",
    ];

    // Construct a Merkle tree with unsorted input
    let tree_1 = MerkleTreeSha256::from_leaves_hashes(
        hashes_1.iter().map(|h| hex::decode(h).unwrap()).collect(),
        false
    ).unwrap();

    // Rearrange nodes by swapping the first two hashes
    let hashes_2 = [
        "713af6f789258961f0ed63b4073060bae6ba7e8b92bcb5383ebe18e6e76289d6",
        "93ca042c86dafda63e1a03b4625f614dc9364231e986d72490a480a4cc591c4a",
        "5d351e5962324a1b9920278825ca07b94d020b34941d20d5ac0f44dbbf3a5258",
        "493c543220bceffa21283b176955173baa7745d563a7b5e2cae0b4253419a87f",
        "39e429c0920f4089a43dbe24a7dfcfe0552bdaabfcc9356cde88f9ea18972bf4",
        "33b544b8002201957eaa0816c4ee2bc244d4cce765e599df6f25adbb2cdb0c08",
        "2e3580210116e2ab7c7705f8f0b6217ac7a0ac5f31de892f847171c073a7542f",
        "016a2b39a42811f88585a5e07ee3a57283607b47816945d61354577ff8868378",
        "c67892017db365f15687b283fea0741145e1b54a62430fd814e1755c6e25949e",
        "af4f1bfe5c512e9265718f3fa9a028f7e70b29860c705763ecbb541f4a5877ce",
        "a0e25bb110b98aa9c3e2be61e432cb432788d0dbe29991acd8ced7c7d9386aea",
        "96ca5d6526e42a2a9da666e27cd8332c3a6b4cada4561726401f66ba08eaaa42",
    ];

    // Construct another tree with rearranged input
    let tree_2 = MerkleTreeSha256::from_leaves_hashes(
        hashes_2.iter().map(|h| hex::decode(h).unwrap()).collect(),
        false
    ).unwrap();

    // Both trees produce the same root hash despite different structures
    assert_eq!(tree_1.get_root().unwrap(), tree_2.get_root().unwrap());
}
```

RECOMMENDATION

Include positional information in hashing, this ensures the hash reflects both the data and its position in the tree, making collisions due to ordering infeasible.

UPDATE

This issue was fixed in commit `6b3bcbb69f2c02b319e4d2ce3cc71933455d143d` .

3.3. Disruption of Merkle Proof Integrity Due to Leaf Sorting [CRITICAL] FIXED

SCOPE

- Commit: `5b6435215eb5ab5d1da57905eae1c5fc1ca0ea27`

DESCRIPTION

The MerkleTree constructor provides an optional sort parameter, which, when enabled, sorts the input leaves before building the tree. While this may be useful for certain applications, it fundamentally alters the order of leaves, directly impacting the tree structure and the resulting root hash.

In the context of Merkle proofs, the integrity and validity of a proof rely on the original order of the leaves. When the leaves are sorted during tree construction, the proof semantics are broken, potentially leading to invalid or misleading proofs.

RECOMMENDATION

Always maintain the original order of the leaves and document any sorting operation outside the tree construction process. Avoid sorting leaves unless required by the application.

UPDATE

This issues was fixed in commit `6b3bcbb69f2c02b319e4d2ce3cc71933455d143d` .

3.4. `get_index_by_hash` is ambiguous for duplicate leaves `[INFO]`

SCOPE

- Commit: `b1c19ef45c28ed67eec5f700a0bc2793088d806a`
- Files: `src/domain/tree/canonical.rs`

DESCRIPTION

The function `get_index_by_hash` is intended to find the tree index for a given leaf hash. However, its current implementation uses `.position()` , which always returns the index of the *first* match it finds.

This behavior is confusing because the library explicitly supports and tests for the presence of duplicate leaves, but provides no way to find the index of any but the first one.

1. This code will only ever find the first occurrence.

```rust
// src/domain/tree/canonical.rs#L49-L54
fn get_index_by_hash(&self, hash: &[u8]) -> Result<usize> {
   self.tree
       .iter()
       .position(|h| h == hash)
       .ok_or_else(|| anyhow!("Hash not found in the Merkle tree"))
}
```

2. The test `test_duplicate_leaves` correctly asserts that multiple identical leaves exist in the tree structure.

```rust
// src/domain/tree/canonical.rs#L215-L240
#[test]
fn test_duplicate_leaves() {
   // If leaves are identical, the tree must contain both at different positions
   let leaves = vec![b"dup".to_vec(), b"dup".to_vec(), b"dup".to_vec()];
   let tree = MerkleTreeCanonicalSha256::from_leaves_data(leaves.clone()).unwrap();
   assert_eq!(
       tree.get_hashes().count(),
       leaves.len(),
       "Tree must have the correct number of leaves"
   );
   // We find all indices for the same hashed leaf
   let dup_hash = Sha256Canonical::hash_leaf(b"dup");
   // Now we search the entire tree for positions
   let positions: Vec<_> = tree
       .tree
       .iter()
       .enumerate()
       .filter_map(|(i, h)| if *h == dup_hash { Some(i) } else { None })
       .collect();
   assert!(
       positions.len() >= 3,
       "We expect at least 3 matches in the array structure"
   );
}
```

UPDATE

The team has been notified of this issue.

4. VERSION HISTORY

Version	Date	Status/Change	Changed by
1.0	*Feb 12, 2026*	Private Report	Verichains Lab